

```

await pool.query('UPDATE users SET dailyCopper = ? WHERE openid = ?', [newDaily, openid]);
return { success: true, dailyCopper: newDaily };
}
async function updateDailyReset(openid, today, resetFields = {}) {
const set = ['lastDailyReset = ?'];
const values = [today.toISOString().slice(0, 10)];
for (const [k, v] of Object.entries(resetFields)) {
if (!['energy', 'dailyCopper'].includes(k)) {
set.push(`${k} = ?`);
values.push(v);
}
}
values.push(openid);
await pool.query('UPDATE users SET ${set.join(', ')} WHERE openid = ?', values);
}
async function updateSettings(openid, settings) {
await pool.query('UPDATE users SET settings = ? WHERE openid = ?', [JSON.stringify(settings), openid]);
}
// ===== 房间 (Redis) =====
async function generateRoomId() {
// 6位大写字母+数字
const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ23456789';
let id;
do {
id = '';
for (let i = 0; i < 6; i++) id += chars[Math.floor(Math.random() * chars.length)];
} while (await redis.exists(K.room + id));
return id;
}
async function createRoom(roomId, creatorOpenid) {
const data = {
roomId,
creatorOpenid,
joinerOpenid: '',
status: 'waiting',
createTime: Date.now(),
expireTime: Date.now() + 5 * 60 * 1000, // 5分钟
};
await redis.set(K.room + roomId, JSON.stringify(data), 'PX', 5 * 60 * 1000);
return data;
}
async function findByRoomId(roomId) {
const raw = await redis.get(K.room + roomId);
return raw ? JSON.parse(raw) : null;
}

```

```

}
async function joinRoom(roomId, joinerOpenid) {
  const raw = await redis.get(K.room + roomId);
  if (!raw) return null;
  const room = JSON.parse(raw);
  room.joinerOpenid = joinerOpenid;
  room.status = 'matched';
  await redis.set(K.room + roomId, JSON.stringify(room), 'PX', 5 * 60 * 1000);
  return room;
}
async function updateStatus(roomId, status) {
  const raw = await redis.get(K.room + roomId);
  if (!raw) return;
  const room = JSON.parse(raw);
  room.status = status;
  await redis.set(K.room + roomId, JSON.stringify(room), 'PX', 5 * 60 * 1000);
}
async function cancelRoom(roomId) {
  await redis.del(K.room + roomId);
}
async function cleanExpired() {
  // Redis 已用 PX 过期，这里为空实现以兼容原接口
  return 0;
}
// ===== 内存态：匹配队列 / 对局 / 掉线 (Redis) =====
async function pushMatchQueue(openid) {
  await redis.rpush(K.queue, openid);
}
async function popMatchQueue() {
  return await redis.lpop(K.queue);
}
async function getQueueLength() {
  return await redis.llen(K.queue);
}
async function removeFromQueue(openid) {
  await redis.lrem(K.queue, 0, openid);
}
async function setGameState(gameId, state) {
  await redis.set(K.game + gameId, JSON.stringify(state), 'PX', 60 * 60 * 1000);
}

```

```
async function getGameState(gameld) {
  const raw = await redis.get(K.game + gameld);
  return raw ? JSON.parse(raw) : null;
}
async function setOfflineCache(openid, data) {
  // 掉线缓存 2 分钟, 用于断线重连
  await redis.set(K.offline + openid, JSON.stringify(data), 'PX', 2 * 60 * 1000);
}
async function getOfflineCache(openid) {
  const raw = await redis.get(K.offline + openid);
  return raw ? JSON.parse(raw) : null;
}
async function clearOfflineCache(openid) {
  await redis.del(K.offline + openid);
}
module.exports = {
  // 用户
  getOrCreateUser,
  findByOpenid,
  updateUser,
  getUserGames,
  getRankList,
  ensureEnergySchema,
  recoverEnergy,
  ensureDailyReset,
  incrementAdCount,
  incrementShareCount,
  // 对局
  createGameRecord,
  updateGameRecord,
  saveGameRecord,
  // 交易
  record,
  getUserTransactions,
  checkin,
  deductEnergy,
  addEnergy,
  updateDailyReset,
  updateSettings,
  // 房间
  generateRoomId,
  createRoom,
  findByRoomId,
  joinRoom,
  updateStatus,
```

```

cancelRoom,
cleanExpired,
// 内存态
pushMatchQueue,
popMatchQueue,
getQueueLength,
removeFromQueue,
setGameState,
getGameState,
setOfflineCache,
getOfflineCache,
clearOfflineCache,
// ===== 兼容命名空间 (session.js / handler.js 调用) =====
userService: {
  getOrCreateUser,
  findByOpenid,
  updateUser,
  getUserGames,
  getRankList,
  // session.js: userService.updateGameRecord(openid, result, ratingChange, afterScore)
  updateGameRecord: updateUserGameResult,
  // handler.js
  addEnergy,
  deductEnergy,
  checkin,
  updateSettings,
  ensureDailyReset,
  incrementAdCount,
  incrementShareCount,
},
gameRecordService: {
  createGameRecord,
  updateGameRecord,
  saveGameRecord,
  getUserGames,
},
transactionService: {
  record,
  getUserTransactions,
},
checkinService: {
  checkin,
},
roomService: {
  // session.js: roomService.createRoom(creatorUid, type)
  async createRoom(creatorUid, type) {
    const roomId = await generateRoomId();
    const data = await createRoom(roomId, creatorUid);
    return { ...data, _id: roomId, type: type || 'invite' };
  }
}

```

```

},
async findByRoomId(roomId) {
const data = await findByRoomId(roomId);
return data ? { ...data, _id: roomId } : null;
},
async joinRoom(roomId, joinerOpenid) {
const data = await joinRoom(roomId, joinerOpenid);
return data ? { ...data, _id: roomId } : null;
},
async updateStatus(roomId, status) {
return updateStatus(roomId, status);
},
async cancelRoom(roomId) {
return cancelRoom(roomId);
},
},
};
===== 文件: server\src\services\session.js =====
/**
 * 六儿, 服务端 — 会话管理服务
 *
 * 管理所有活跃的 WebSocket 连接和对局会话
 * 维护:
 * - wsMap: openid → WebSocket 连接的映射
 * - gameSessions: gameId → GameEngine 实例的映射
 * - matchingQueue: 匹配队列
 * - roomMap: roomId → 房间信息的映射
 */
const GameEngine = require('../game/engine');
const { Stage } = require('../game/constants');
const { userService, gameRecordService, roomService, transactionService } = require('../data');
const { game: gameConfig } = require('../config');
// ===== 全局状态容器 =====
/** openid → { ws, openid, nickName, avatarUrl, user: 用户数据 } */
const wsMap = new Map();
/** gameId → GameEngine */
const gameSessions = new Map();
/** 匹配队列: [{ openid, nickName, avatarUrl, rankScore }] */
const matchingQueue = [];
/** roomId → { roomDocId, creatorWs, joinerWs, creatorUid, joinerUid } */
const roomMap = new Map();
/** 生成对局ID (进程内计数器 + 时间戳 + 随机 + pid, 避免并发碰撞) */
let _gameSeq = 0;

```

```

function generateGameId() {
  _gameSeq = (_gameSeq + 1) % 1000000;
  return `G${Date.now()}-${process.pid}-${Math.random().toString(36).slice(2, 8)}-${(_gameSeq).toString().padStart(6, '0')}`;
}
// ===== 连接管理 =====
/** 注册连接 */
function registerConnection(ws, openid, userData) {
  wsMap.set(openid, {
    ws,
    openid,
    nickName: userData.nickName || "",
    avatarUrl: userData.avatarUrl || "",
    user: userData,
  });
}
/** 移除连接 */
function removeConnection(openid) {
  // 如果正在匹配队列中, 移除
  const idx = matchingQueue.findIndex(p => p.openid === openid);
  if (idx !== -1) matchingQueue.splice(idx, 1);
  // 如果在某个游戏会话中, 触发掉线处理
  const activeGame = findActiveGameByPlayer(openid);
  if (activeGame) {
    handlePlayerDisconnect(activeGame, openid);
  }
  wsMap.delete(openid);
}
/** 根据 openid 发送消息 */
function sendToPlayer(openid, msg) {
  const session = wsMap.get(openid);
  if (session && session.ws.readyState === 1) { // WebSocket.OPEN
    session.ws.send(JSON.stringify(msg));
  }
}
/** 广播给对局双方 */
function broadcastToGame(gameId, msg) {
  const engine = gameSessions.get(gameId);
  if (!engine) return;
  sendToPlayer(engine.blackPlayer.openid, msg);
  sendToPlayer(engine.whitePlayer.openid, msg);
}
/** 查找玩家所在的活跃对局 */

```

```

function findActiveGameByPlayer(openid) {
  for (const [gameId, engine] of gameSessions) {
    if (engine.stage !== Stage.SETTLED) {
      if (engine.blackPlayer.openid === openid || engine.whitePlayer.openid === openid) {
        return { gameId, engine };
      }
    }
  }
  return null;
}

/** 获取用户的对局对手 openid */
function getOpponentUid(engine, openid) {
  if (engine.blackPlayer.openid === openid) return engine.whitePlayer.openid;
  if (engine.whitePlayer.openid === openid) return engine.blackPlayer.openid;
  return null;
}

// ===== 匹配系统 =====
/** 加入匹配队列 */
async function joinMatching(player) {
  // 检查是否已在队列中
  if (matchingQueue.find(p => p.openid === player.openid)) {
    return { success: false, errMsg: '已在匹配队列中' };
  }
  // 检查是否已在游戏中
  if (findActiveGameByPlayer(player.openid)) {
    return { success: false, errMsg: '你正在对局中' };
  }
  matchingQueue.push(player);
  sendToPlayer(player.openid, {
    cmd: 'match_status',
    data: { status: 'matching', queueSize: matchingQueue.length },
  });
  // 尝试匹配
  await tryMatch();
  return { success: true, queueSize: matchingQueue.length };
}

/** 取消匹配 */
function cancelMatching(openid) {
  const idx = matchingQueue.findIndex(p => p.openid === openid);
  if (idx !== -1) {
    matchingQueue.splice(idx, 1);
    sendToPlayer(openid, { cmd: 'match_status', data: { status: 'cancelled' } });
    return { success: true };
  }
}

```

```

}
return { success: false, errMsg: '不在匹配队列中' };
}
/** 尝试匹配 */
async function tryMatch() {
  while (matchingQueue.length >= 2) {
    const player1 = matchingQueue.shift();
    const player2 = matchingQueue.shift();
    // 随机分配黑白
    const [blackPlayer, whitePlayer] = Math.random() < 0.5
      ? [player1, player2]
      : [player2, player1];
    await startGame(blackPlayer, whitePlayer, 'random');
  }
}
// ===== 房间系统 =====
/** 创建邀请房间 */
async function createInviteRoom(creatorUid) {
  const room = await roomService.createRoom(creatorUid, 'invite');
  roomMap.set(room.roomId, {
    roomDocId: room._id,
    creatorUid,
    joinerUid: '',
  });
  sendToPlayer(creatorUid, {
    cmd: 'room_created',
    data: { roomId: room.roomId },
  });
  // 超时自动解散
  setTimeout(() => {
    const rm = roomMap.get(room.roomId);
    if (rm && !rm.joinerUid) {
      sendToPlayer(rm.creatorUid, {
        cmd: 'room_expired',
        data: { roomId: room.roomId },
      });
      roomService.cancelRoom(rm.roomDocId);
      roomMap.delete(room.roomId);
    }
  }, gameConfig.roomExpire);
  return { success: true, roomId: room.roomId };
}

```



```

/** 通过房间号加入 */
async function joinRoomByCode(joinerUid, roomId) {
  const room = await roomService.findByRoomId(roomId);
  if (!room) {
    return { success: false, errMsg: '房间不存在或已过期' };
  }
  // 阻断: 不能加入自己创建的房间 (否则双方为同一人, 对局异常)
  if (room.creatorOpenid && room.creatorOpenid === joinerUid) {
    return { success: false, errMsg: '不能进入自己创建的房间' };
  }
  await roomService.joinRoom(room._id, joinerUid);
  const rm = roomMap.get(roomId);
  if (rm) {
    rm.joinerUid = joinerUid;
  }
  // 通知双方 (数据库房间字段为 creatorOpenid)
  const creatorUid = room.creatorOpenid;
  const creatorSession = wsMap.get(creatorUid);
  const joinerSession = wsMap.get(joinerUid);
  console.log('[Room] joinRoomByCode: joiner=${joinerUid}, creator=${creatorUid}, creatorSession=${!!creatorSession}, joinerSession=${!!joinerSession}');
  const creatorUser = creatorSession ? creatorSession.user : {};
  const joinerUser = joinerSession ? joinerSession.user : {};
  sendToPlayer(creatorUid, {
    cmd: 'opponent_joined',
    data: {
      roomId,
      opponent: {
        openid: joinerUid,
        nickName: joinerUser.nickName || '',
        avatarUrl: joinerUser.avatarUrl || '',
        rankScore: joinerUser.rankScore || 0,
      },
    },
  });
  sendToPlayer(joinerUid, {
    cmd: 'join_room_success',
    data: {
      roomId,
      opponent: {
        openid: creatorUid,
        nickName: creatorUser.nickName || '',
        avatarUrl: creatorUser.avatarUrl || '',

```

```

rankScore: creatorUser.rankScore || 0,
},
},
});
// 双方就位, 自动开始对局
const creatorPlayer = {
  openid: creatorUid,
  nickName: creatorUser.nickName || '',
  avatarUrl: creatorUser.avatarUrl || '',
  rankScore: creatorUser.rankScore || 0,
};
const joinerPlayer = {
  openid: joinerUid,
  nickName: joinerUser.nickName || '',
  avatarUrl: joinerUser.avatarUrl || '',
  rankScore: joinerUser.rankScore || 0,
};
// 随机分配黑白
const [blackPlayer, whitePlayer] = Math.random() < 0.5
? [creatorPlayer, joinerPlayer]
: [joinerPlayer, creatorPlayer];
await startGame(blackPlayer, whitePlayer, 'room');
// 清理房间
if (rm) {
  roomMap.delete(roomId);
}
roomService.cancelRoom(room._id);
return { success: true };
}
// ===== 退出房间 =====
/** 房主或加入者退出房间, 通知对方并清理 */
async function leaveRoom(uid, roomId) {
  let rm = null;
  if (roomId) {
    rm = roomMap.get(roomId);
  } else {
    // 未带 roomId 时按 uid 反查
    for (const [rid, r] of roomMap.entries()) {
      if (r.creatorUid === uid || r.joinerUid === uid) { rm = r; roomId = rid; break; }
    }
  }
  if (rm) {
    const otherUid = rm.creatorUid === uid ? rm.joinerUid : rm.creatorUid;
    if (otherUid) {

```

```

sendToPlayer(otherUid, { cmd: 'room_cancelled', data: { roomId } });
}
try { roomService.cancelRoom(rm.roomDocId); } catch (e) { /* ignore */ }
roomMap.delete(roomId);
}
return { success: true };
}
// ===== 对局管理 =====
/** 开始对局 */
async function startGame(blackPlayer, whitePlayer, roomType) {
const gameId = generateGameId();
// 开局消耗双方精力 (每局 energyPerGame) , 不足则跳过扣减, 避免对局卡死
const cost = gameConfig.energyPerGame || 1;
try {
await userService.deductEnergy(blackPlayer.openid, cost);
await userService.deductEnergy(whitePlayer.openid, cost);
} catch (e) {
console.error('[Session] 扣除开局精力失败:', e);
}
const engine = new GameEngine(gameId, blackPlayer, whitePlayer);
engine.init();
gameSessions.set(gameId, engine);
// 通知双方游戏开始
const startMsg = {
cmd: 'game_start',
data: {
gameId,
stage: Stage.PLACING,
currentTurn: engine.currentTurn, // WHITE 先手
remainingTime: Math.ceil(gameConfig.moveTimeout / 1000),
board: engine.board,
blackPlayer: {
openid: blackPlayer.openid,
nickName: blackPlayer.nickName,
avatarUrl: blackPlayer.avatarUrl,
rankScore: blackPlayer.rankScore,
},
whitePlayer: {
openid: whitePlayer.openid,
nickName: whitePlayer.nickName,
avatarUrl: whitePlayer.avatarUrl,
rankScore: whitePlayer.rankScore,
},
timeLimit: gameConfig.moveTimeout,
},
},

```

```

};
broadcastToGame(gameld, startMsg);
// 启动第一回合超时
engine.startTurnTimer((color) => {
  handleTimeout(gameld, color);
});
}
/** 处理回合超时 */
function handleTimeout(gameld, color) {
  const engine = gameSessions.get(gameld);
  if (!engine || engine.stage === Stage.SETTLED) return;
  const result = engine.autoTimeout(color);
  const openid = color === 1 ? engine.blackPlayer.openid : engine.whitePlayer.openid;
  // 仅通知当前超时的那一方 (对方无需提示)
  sendToPlayer(openid, {
    cmd: 'timeout_warning',
    data: {
      openid,
      player: color,
      nickName: engine.getPlayerByColor(color).nickName || '',
      consecutiveTimeouts: result.consecutiveTimeouts,
    },
  });
  // 广播操作结果
  broadcastToGame(gameld, buildBroadcastMsg(engine, result, openid));
  // 如果结算
  if (result.settled) {
    finalizeGame(gameld, result);
    return;
  }
  // 启动下一回合超时
  engine.startTurnTimer((c) => {
    handleTimeout(gameld, c);
  });
}
/** 处理玩家掉线 */
function handlePlayerDisconnect({ gameld, engine }, openid) {
  // 防止重复触发 (close + 心跳超时可能同时触发)
  const disconnectKey = `${gameld}:${openid}`;
  if (engine._disconnectTimers && engine._disconnectTimers.has(disconnectKey)) return;

```

```

if (!engine._disconnectTimers) engine._disconnectTimers = new Set();
engine._disconnectTimers.add(disconnectKey);
// 通知对手
const opponentUid = getOpponentUid(engine, openid);
sendToPlayer(opponentUid, {
  cmd: 'opponent_disconnected',
  data: { openid },
});
console.log('[Session] 玩家掉线启动重连窗口: ${openid}, game=${gameld}');
// 设置重连窗口
setTimeout(() => {
  engine._disconnectTimers.delete(disconnectKey);
  // 检查玩家是否已重连
  const stillConnected = wsMap.has(openid);
  if (!stillConnected && engine.stage !== Stage.SETTLED) {
    console.log('[Session] 重连窗口到期, 判掉线方负: ${openid}');
    // 判掉线方负
    const color = engine.getColorByUid(openid);
    const winner = color === 1 ? 2 : 1;
    const result = engine.settleGame(
      winner === 1 ? 'black' : 'white',
      'disconnect'
    );
    broadcastToGame(gameld, {
      cmd: 'game_settle',
      data: result,
    });
    finalizeGame(gameld, result);
  }
}, gameConfig.reconnectWindow);
}
// ===== 消息处理 =====
/** 处理玩家操作消息 */
async function handleGameAction(openid, cmd, data) {
  const activeGame = findActiveGameByPlayer(openid);
  if (!activeGame) {
    sendToPlayer(openid, { cmd: 'error', data: { errMsg: '未找到进行中的对局, 可能已结束或服务器已重启' } });
    return;
  }
  const { gameld, engine } = activeGame;
  let result;

```

```

switch (cmd) {
case 'place_piece':
result = engine.placePiece(openid, data.r, data.c);
break;
case 'capture_piece':
result = engine.capturePiece(openid, data.r, data.c);
break;
case 'move_piece':
result = engine.movePiece(openid, data.fromR, data.fromC, data.toR, data.toC);
break;
case 'linked_capture':
result = engine.linkedCapturePiece(openid, data.r, data.c);
break;
case 'skip_capture':
result = engine.skipLinkedCapture(openid);
break;
case 'give_up':
result = engine.surrender(openid);
if (result.success && result.settled) {
broadcastToGame(gameId, { cmd: 'game_settle', data: result });
finalizeGame(gameId, result);
return;
}
break;
case 'request_draw':
result = engine.requestDraw(openid);
if (result.success && result.drawRequestBy) {
const opponentUid = getOpponentUid(engine, openid);
sendToPlayer(opponentUid, {
cmd: 'draw_request',
data: {
by: openid,
nickName: wsMap.get(openid)?.nickName || "",
},
});
// 求和请求已处理, 不重启计时器 (游戏暂停等待响应)
return;
}
break;
case 'respond_draw':
result = engine.respondDraw(openid, data.agree);
if (result.success && result.settled) {
broadcastToGame(gameId, { cmd: 'game_settle', data: result });
}
}

```

```

finalizeGame(gameId, result);
return;
}
break;
default:
result = { success: false, errMsg: `未知指令: ${cmd}` };
}
if (!result.success) {
sendToPlayer(openid, { cmd: 'error', data: { errMsg: result.errMsg || '操作失败' } });
return;
}
if (result.settled) {
// 操作直接导致结算（绝杀/联动揪光/和棋等），广播并保存战绩
broadcastToGame(gameId, { cmd: 'game_settle', data: result });
finalizeGame(gameId, result);
return;
}
// 清除同方向的操作超时
const color = engine.getColorByUid(openid);
engine.consecutiveTimeouts[color] = 0;
// 广播操作结果
broadcastToGame(gameId, buildBroadcastMsg(engine, result, openid));
// 启动下一回合超时
if (engine.stage !== Stage.SETTLED) {
engine.clearTimer();
engine.startTurnTimer((c) => {
handleTimeout(gameId, c);
});
}
}
/** 构建广播消息 */
function buildBroadcastMsg(engine, result, openid) {
const color = engine.getColorByUid(openid);
const msg = {
cmd: '',
data: {
stage: engine.stage,
currentTurn: engine.currentTurn,
remainingTime: Math.ceil(engine.getRemainingTime() / 1000),
board: result.board || engine.board,
lastMove: {
player: color,

```

```

action: result.lastAction,
},
},
};
if (result.stageChanged) {
  msg.cmd = 'stage_change';
  msg.data.stage = result.stage;
  msg.data.catchNums = result.catchNums;
} else if (result.linkedCapture) {
  msg.cmd = 'linked_capture';
  msg.data.linkedCapture = result.linkedCapture;
} else if (result.drawRejected) {
  msg.cmd = 'draw_rejected';
} else if (result.noNewForm) {
  msg.cmd = 'move_made';
} else if (result.drawRequestBy) {
  msg.cmd = 'draw_requested';
} else {
  // 依据引擎返回的动作类型决定广播指令，避免下子被误判为揪子
  switch (result.lastAction) {
    case 'capture':
      msg.cmd = 'capture_made';
      break;
    case 'move':
      msg.cmd = 'move_made';
      break;
    case 'place':
      default:
      msg.cmd = 'piece_placed';
      break;
  }
}
msg.data.catchNums = result.catchNums || {
  black: engine.blackCatchNum,
  white: engine.whiteCatchNum,
};
return msg;
}
// ===== 对局结算 =====
async function finalizeGame(gameId, settleResult) {
  const engine = gameSessions.get(gameId);
  if (!engine) return;
  engine.clearTimer();

```



```

// 更新双方战绩
const blackResult = settleResult.result === 'black' ? 'win'
: settleResult.result === 'white' ? 'lose' : 'draw';
const whiteResult = settleResult.result === 'white' ? 'win'
: settleResult.result === 'black' ? 'lose' : 'draw';
await Promise.all([
  userService.updateGameRecord(engine.blackPlayer.openid, blackResult, settleResult.blackRatingChange, settleResult.blackAfterScore),
  userService.updateGameRecord(engine.whitePlayer.openid, whiteResult, settleResult.whiteRatingChange, settleResult.whiteAfterScore),
]);
// 从引擎计算真实统计值 (settleResult 不携带这些字段)
const allMoves = engine.moves || [];
const blackMoves = allMoves.filter(m => m.stage === Stage.MOVING && m.player === 'black').length;
const whiteMoves = allMoves.filter(m => m.stage === Stage.MOVING && m.player === 'white').length;
const blackCaptures = allMoves.filter(m => m.stage === Stage.CAPTURING && m.player === 'black').length;
const whiteCaptures = allMoves.filter(m => m.stage === Stage.CAPTURING && m.player === 'white').length;
const durationMs = (engine.endedAt && engine.startedAt)
? (engine.endedAt - engine.startedAt)
: 0;
// 保存对局记录
await gameRecordService.saveGameRecord({
  gameId,
  ...settleResult,
  blackPlayer: engine.blackPlayer,
  whitePlayer: engine.whitePlayer,
  blackMoves,
  whiteMoves,
  blackCaptures,
  whiteCaptures,
  durationMs,
});
// 资源变更通知 (精力已含自然恢复时间戳, 客户端据此显示"下次恢复"倒计时)
const freshBlackUser = await userService.findByOpenid(engine.blackPlayer.openid);
const freshWhiteUser = await userService.findByOpenid(engine.whitePlayer.openid);
sendToPlayer(engine.blackPlayer.openid, {
  cmd: 'resource_update',
  data: {
    rankScore: freshBlackUser?.rankScore || 0,
    rankName: freshBlackUser?.rankName || '初级小六',
    energy: freshBlackUser?.energy || 0,
    energyRecoverAt: freshBlackUser?.energyRecoverAt || 0,
    energyMax: gameConfig.energyMax || 30,
  },
});
sendToPlayer(engine.whitePlayer.openid, {

```

```

cmd: 'resource_update',
data: {
  rankScore: freshWhiteUser?.rankScore || 0,
  rankName: freshWhiteUser?.rankName || '初级小六',
  energy: freshWhiteUser?.energy || 0,
  energyRecoverAt: freshWhiteUser?.energyRecoverAt || 0,
  energyMax: gameConfig.energyMax || 30,
},
});
}
// ===== 导出 =====
module.exports = {
  wsMap,
  gameSessions,
  matchingQueue,
  roomMap,
  registerConnection,
  removeConnection,
  sendToPlayer,
  broadcastToGame,
  findActiveGameByPlayer,
  joinMatching,
  cancelMatching,
  createInviteRoom,
  joinRoomByCode,
  leaveRoom,
  startGame,
  handleGameAction,
  handlePlayerDisconnect,
  finalizeGame,
  generateGameId,
};
===== 文件: server\src\ws\handler.js =====
/**
 * 六儿 服务端 — WebSocket 消息路由
 *
 * 协议格式 (PRD 定义) :
 * {
 *   "cmd": "指令名",
 *   "data": {},
 *   "seq": 序号 (可选)
 * }
 */
const { userService, transactionService, checkinService, gameRecordService } = require('../services/data');
const {
  wsMap, registerConnection, removeConnection,
  sendToPlayer, findActiveGameByPlayer,

```

```

joinMatching, cancelMatching,
createInviteRoom, joinRoomByCode,
handleGameAction, gameSessions,
} = require('../services/session');
const { getRankName } = require('../game/board');
const { game: gameConfig } = require('../config');
/**
 * 主消息分发器
 * @param {WebSocket} ws
 * @param {object} msg - { cmd, data, seq }
 */
async function dispatch(ws, msg) {
  const { cmd, data = {}, seq } = msg;
  try {
    // ===== 认证 =====
    // login 是异步的（要查 DB）。服务端按消息顺序收到 login 后，把它包装成 Promise；
    // 后续非 login 指令必须先 await 该 Promise，避免在 DB 查询期间并发处理业务指令，
    // 导致连接尚未注册到 wsMap 就返回"请先登录"。
    if (cmd === 'login') {
      ws.loginPromise = handleLogin(ws, data).catch((err) => {
        console.error('[WS] handleLogin 异常:', err);
      });
      await ws.loginPromise;
      return;
    }
    // ===== 心跳 =====
    // ping 不需要登录，直接响应
    if (cmd === 'ping') {
      ws.isAlive = true;
      ws.send(JSON.stringify({ cmd: 'pong' }));
      return;
    }
    // ===== 其他业务指令 =====
    // 若登录仍在进行中，等待登录完成
    if (ws.loginPromise) {
      await ws.loginPromise;
    }
    const openid = getOpenid(ws);
    if (!openid) {
      console.log('[WS] 未登录请求被拒绝: cmd=${cmd}, ws.openid=${ws.openid || 'null'}, wsMap.has=${wsMap.has(ws.openid || '')});
      safeSend(ws, {
        cmd: 'error',
        data: { errMsg: '请先登录', cmd },
        seq,
      });
    }
  }
}

```

```
return;
}
switch (cmd) {
// ===== 匹配 =====
case 'match_start':
    handleMatchStart(ws, data);
    break;
case 'match_cancel':
    handleMatchCancel(ws);
    break;
// ===== 房间 =====
case 'invite_room':
    await handleInviteRoom(ws);
    break;
case 'join_room':
    await handleJoinRoom(ws, data);
    break;
case 'leave_room':
    await handleLeaveRoom(ws, data);
    break;
// ===== 对局操作 =====
case 'place_piece':
case 'capture_piece':
case 'move_piece':
case 'linked_capture':
case 'skip_capture':
case 'give_up':
case 'request_draw':
case 'respond_draw':
    handleGameAction(openid, cmd, data);
    break;
// ===== 用户数据 =====
case 'get_profile':
    await handleGetProfile(ws);
    break;
case 'get_history':
    await handleGetHistory(ws, data);
    break;
case 'update_settings':
    await handleUpdateSettings(ws, data);
    break;
case 'update_profile':
    await handleUpdateProfile(ws, data);
    break;
// ===== 排行榜 =====
```

```

case 'get_rank_list':
await handleGetRankList(ws, data);
break;
// ===== 经济系统 =====
case 'sign_in':
await handleSignIn(ws);
break;
case 'get_transactions':
await handleGetTransactions(ws, data);
break;
case 'get_ad_reward':
await handleAdReward(ws);
break;
case 'get_share_reward':
await handleShareReward(ws);
break;
// ===== 重连 =====
case 'reconnect':
handleReconnect(ws, data);
break;
default:
sendToPlayer(openid, {
cmd: 'error',
data: { errMsg: `未知指令: ${cmd}` },
});
} catch (err) {
console.error(`[WS] 处理指令 ${cmd} 出错:`, err);
safeSend(ws, {
cmd: 'error',
data: { errMsg: '服务器内部错误', cmd },
seq,
});
}
}
// ===== 辅助 =====
/** 获取 WebSocket 对应的 openid */
function getOpenid(ws) {
// 优先用 ws 上已绑定的 openid 直接查 wsMap, 避免同 openid 多连接时
// session.ws 引用比较失效导致的“请先登录” (ws.openid 在 handleLogin 中已设置)。
if (ws && ws.openid && wsMap.has(ws.openid)) {
return ws.openid;
}
for (const [openid, session] of wsMap) {
if (session.ws === ws) return openid;
}
}

```

```

}
return null;
}
// ===== 认证处理 =====
async function handleLogin(ws, data) {
  const { openid, nickName, avatarUrl } = data;
  if (!openid) {
    ws.send(JSON.stringify({ cmd: 'error', data: { errMsg: '缺少 openid 参数' } }));
    return;
  }
  // ★ 关键：先发 login_ack 确认收到登录请求（不等 DB）
  // 这样 CB 代理层知道连接活跃，防止 1006
  safeSend(ws, { cmd: 'login_ack', data: { openid } });
  console.log(`[WS] 登录请求: ${openid}, 开始查询数据库...`);
  // ===== DB 操作独立 try-catch，失败不影响连接 =====
  try {
    // 获取或创建用户（带 5s 超时）
    const user = await Promise.race([
      userService.getOrCreateUser(openid, { nickName, avatarUrl }),
      new Promise(., reject) =>
        setTimeout(() => reject(new Error('数据库查询超时(5s)'), 5000)
        ),
    ]);
    // 注册连接，并把 openid 绑定到 ws 对象方便心跳/close 直接定位
    ws.openid = openid;
    registerConnection(ws, openid, user);
    // 返回用户数据
    safeSend(ws, {
      cmd: 'login_success',
      data: {
        openid,
        nickName: user.nickName,
        avatarUrl: user.avatarUrl,
        rankScore: user.rankScore,
        rankName: user.rankName,
        // 场次统计：getOrCreateUser 返回 winCount/loseCount/drawCount（wins/losses/draws 为旧字段，兼容保留）
        totalGames: ((user.winCount || 0) + (user.loseCount || 0) + (user.drawCount || 0)),
        winCount: user.winCount || 0,
        loseCount: user.loseCount || 0,
        drawCount: user.drawCount || 0,
        wins: user.winCount || 0,
        losses: user.loseCount || 0,
        draws: user.drawCount || 0,
      },
    });
  } catch (err) {
    console.error('登录失败:', err);
    ws.send(JSON.stringify({ cmd: 'error', data: { errMsg: '登录失败' } }));
  }
}

```

```

winRate: user.winRate,
energy: user.energy,
energyMax: user.energyMax,
energyRecoverAt: user.energyRecoverAt || 0,
dailyAdCount: user.dailyAdCount || 0,
dailyShareCount: user.dailyShareCount || 0,
regretCards: user.regretCards,
renameCards: user.renameCards,
settings: user.settings,
signinStreak: user.signinStreak,
lastCheckin: user.lastCheckin,
},
});
console.log('[WS] 用户登录成功: ${openid}');
} catch (dbErr) {
console.error('[WS] 登录 DB 错误 (${openid}):', dbErr.message);
// ★ 使用降级用户数据 (纯内存, 不走 DB)
const fallbackUser = createFallbackUser(openid, nickName, avatarUrl);
registerConnection(ws, openid, fallbackUser);
safeSend(ws, {
cmd: 'login_success',
data: {
openid,
nickName: fallbackUser.nickName,
avatarUrl: fallbackUser.avatarUrl,
rankScore: fallbackUser.rankScore,
rankName: fallbackUser.rankName,
totalGames: 0, winCount: 0, loseCount: 0, drawCount: 0, wins: 0, losses: 0, draws: 0, winRate: 0,
energy: 30, energyMax: 30, energyRecoverAt: 0,
regretCards: 0, renameCards: 0,
settings: { soundEnabled: true, vibrationEnabled: true, musicEnabled: true },
signinStreak: 0, lastCheckin: "",
},
});
console.log('[WS] 用户降级登录: ${openid} (DB不可用)');
}
}
// ===== 匹配处理 =====
function handleMatchStart(ws, data) {
const openid = getOpenid(ws);
if (!openid) {
ws.send(JSON.stringify({ cmd: 'error', data: { errMsg: '请先登录' } }));
return;
}
}

```

```

const session = wsMap.get(openid);
if (!session) return;
const result = joinMatching({
  openid,
  nickName: session.nickName,
  avatarUrl: session.avatarUrl,
  rankScore: session.user?.rankScore || 0,
});
if (!result.success) {
  sendToPlayer(openid, { cmd: 'error', data: { errMsg: result.errMsg } });
}
}
function handleMatchCancel(ws) {
  const openid = getOpenid(ws);
  if (!openid) return;
  cancelMatching(openid);
}
// ===== 房间处理 =====
async function handleInviteRoom(ws) {
  const openid = getOpenid(ws);
  if (!openid) {
    ws.send(JSON.stringify({ cmd: 'error', data: { errMsg: '请先登录' } }));
    return;
  }
  await createInviteRoom(openid);
}
async function handleJoinRoom(ws, data) {
  const openid = getOpenid(ws);
  if (!openid) {
    ws.send(JSON.stringify({ cmd: 'error', data: { errMsg: '请先登录' } }));
    return;
  }
  const { roomId } = data;
  if (!roomId) {
    sendToPlayer(openid, { cmd: 'error', data: { errMsg: '请输入房间号' } });
    return;
  }
  const res = await joinRoomByCode(openid, roomId);
  if (!res || !res.success) {
    sendToPlayer(openid, { cmd: 'error', data: { errMsg: (res && res.errMsg) || '加入房间失败' } });
  }
}

```



```
async function handleLeaveRoom(ws, data) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const roomId = data && data.roomId;
  await leaveRoom(openid, roomId);
}
// ===== 用户数据处理 =====
async function handleGetProfile(ws) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const user = await userService.findByOpenid(openid);
  if (!user) {
    sendToPlayer(openid, { cmd: 'error', data: { errMsg: '用户不存在' } });
    return;
  }
  sendToPlayer(openid, {
    cmd: 'profile',
    data: {
      openid: user._openid,
      nickName: user.nickName,
      avatarUrl: user.avatarUrl,
      rankScore: user.rankScore,
      rankName: user.rankName,
      totalGames: user.totalGames,
      wins: user.wins,
      losses: user.losses,
      draws: user.draws,
      winRate: user.winRate,
      maxRankScore: user.maxRankScore,
      energy: user.energy,
      energyMax: user.energyMax,
      regretCards: user.regretCards,
      renameCards: user.renameCards,
      settings: user.settings,
      signinStreak: user.signinStreak,
      lastSignInDate: user.lastSignInDate,
      createdAt: user.createdAt,
    },
  });
}
async function handleGetHistory(ws, data) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const limit = data.limit || 20;
```

```

const games = await gameRecordService.getUserGames(openid, limit);
sendToPlayer(openid, {
  cmd: 'history',
  data: { games },
});
}
async function handleUpdateSettings(ws, data) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const result = await userService.updateSettings(openid, data.settings || {});
  if (result.success) {
    sendToPlayer(openid, { cmd: 'settings_updated', data: { settings: result.settings } });
  } else {
    sendToPlayer(openid, { cmd: 'error', data: { errMsg: result.errMsg } });
  }
}
// 更新昵称/头像 ("完善资料"浮层提交)
async function handleUpdateProfile(ws, data) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const updates = {};
  if (typeof data.nickName === 'string' && data.nickName.trim()) {
    updates.nickName = data.nickName.trim().slice(0, 30);
  }
  if (typeof data.avatarUrl === 'string' && data.avatarUrl.trim()) {
    updates.avatarUrl = data.avatarUrl.trim().slice(0, 512);
  }
  if (!Object.keys(updates).length) {
    return sendToPlayer(openid, { cmd: 'error', data: { errMsg: '没有可更新的数据' } });
  }
  try {
    await userService.updateUser(openid, updates);
    const user = await userService.findByOpenid(openid);
    sendToPlayer(openid, {
      cmd: 'profile_updated',
      data: {
        openid,
        nickName: user ? user.nickName : updates.nickName,
        avatarUrl: user ? user.avatarUrl : updates.avatarUrl,
      },
    });
  } catch (err) {
    console.error('[WS] 更新资料失败:', err);
    sendToPlayer(openid, { cmd: 'error', data: { errMsg: '资料保存失败' } });
  }
}

```

```

}
}
// ===== 排行榜 =====
async function handleGetRankList(ws, data) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const sortBy = data.type === 'winRate' ? 'winRate' : 'score';
  const limit = data.limit || 100;
  const rankList = await userService.getRankList(limit, sortBy);
  // 计算自己的排名 (按相同规则查询更大范围后定位)
  let myRank = -1;
  let myWinRate = 0;
  const user = await userService.findByOpenid(openid);
  if (user) {
    if (sortBy === 'winRate') {
      const total = (user.winCount || 0) + (user.loseCount || 0) + (user.drawCount || 0);
      myWinRate = total > 0 ? Math.round((user.winCount || 0) * 1000 / total) / 10 : 0;
    }
    // 用较大 limit 查询真实名次 (避免仅在 rankList 内查找漏判)
    const fullList = await userService.getRankList(500, sortBy);
    myRank = fullList.findIndex(r => r.openid === openid) + 1;
  }
  sendToPlayer(openid, {
    cmd: 'rank_list',
    data: {
      rankList,
      myRank,
      myWinRate,
      myRankScore: user?.rankScore || 0,
      myRankName: user?.rankName || '初级小六',
      myTotalGames: user ? ((user.winCount || 0) + (user.lossCount || 0) + (user.drawCount || 0)) : 0,
      sortBy,
    },
  });
}
// ===== 经济系统 =====
async function handleSignIn(ws) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const result = await checkinService.checkin(openid);
  if (result.success) {
    // 签到奖励精力 (不再发放铜板)
  }
}

```

```

const energyRes = await userService.addEnergy(openid, result.bonus || 5);
sendToPlayer(openid, {
  cmd: 'sign_in_result',
  data: {
    streak: result.streak,
    dayIndex: result.dayIndex,
    energy: energyRes.energy,
  },
});
sendToPlayer(openid, {
  cmd: 'resource_update',
  data: { energy: energyRes.energy, energyRecoverAt: energyRes.energyRecoverAt || 0 },
});
} else {
  sendToPlayer(openid, { cmd: 'error', data: { errMsg: result.errMsg } });
}
}
}
async function handleGetTransactions(ws, data) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const limit = data.limit || 50;
  const transactions = await transactionService.getUserTransactions(openid, limit);
  sendToPlayer(openid, {
    cmd: 'transactions',
    data: { transactions },
  });
}
}
async function handleAdReward(ws) {
  const openid = getOpenid(ws);
  if (!openid) return;
  const maxAd = gameConfig.maxAdPerDay || 3;
  // 跨天重置每日计数
  const reset = await userService.ensureDailyReset(openid);
  const cur = reset && reset.success ? (reset.adCount || 0) : 0;
  if (cur >= maxAd) {
    sendToPlayer(openid, { cmd: 'error', data: { errMsg: '今日看视频次数已用完' } });
    return;
  }
  const reward = gameConfig.adReward || 10;
  const result = await userService.addEnergy(openid, reward);
  if (result.success) {
    const inc = await userService.incrementAdCount(openid);
    sendToPlayer(openid, {
      cmd: 'ad_reward_result',

```

```

data: { energy: result.energy, reward, adCount: inc && inc.count ? inc.count : 0 },
});
sendToPlayer(openid, {
cmd: 'resource_update',
data: { energy: result.energy, energyRecoverAt: result.energyRecoverAt || 0 },
});
}
}
async function handleShareReward(ws) {
const openid = getOpenid(ws);
if (!openid) return;
const maxShare = gameConfig.shareRewardLimit || 5;
const reset = await userService.ensureDailyReset(openid);
const cur = reset && reset.success ? (reset.shareCount || 0) : 0;
if (cur >= maxShare) {
sendToPlayer(openid, { cmd: 'error', data: { errMsg: '今日分享次数已用完' } });
return;
}
const reward = gameConfig.shareReward || 5;
const result = await userService.addEnergy(openid, reward);
if (result.success) {
const inc = await userService.incrementShareCount(openid);
sendToPlayer(openid, {
cmd: 'share_reward_result',
data: { energy: result.energy, reward, shareCount: inc && inc.count ? inc.count : 0 },
});
sendToPlayer(openid, {
cmd: 'resource_update',
data: { energy: result.energy, energyRecoverAt: result.energyRecoverAt || 0 },
});
}
}
// ===== 重连处理 =====
function handleReconnect(ws, data) {
const openid = getOpenid(ws);
if (!openid) return;
const activeGame = findActiveGameByPlayer(openid);
if (activeGame) {
const { gameId, engine } = activeGame;
// 更新 WebSocket 引用
const session = wsMap.get(openid);
if (session) session.ws = ws;
// 发送游戏快照

```

```

ws.send(JSON.stringify({
  cmd: 'game_snapshot',
  data: engine.getSnapshot(),
}));
}
}
module.exports = { dispatch };
// ===== 辅助工具函数 =====
/** 安全发送消息, 忽略连接已关闭错误 */
function safeSend(ws, msg) {
  try {
    if (ws.readyState === 1) { // WebSocket.OPEN
      ws.send(JSON.stringify(msg));
    }
  } catch (err) {
    console.error('[WS] safeSend 失败:', err.message);
  }
}
/** 创建降级用户数据 (DB 不可用时使用纯内存版本) */
function createFallbackUser(openid, nickName, avatarUrl) {
  const now = Date.now();
  return {
    _openid: openid,
    _id: 'fallback_' + openid,
    nickName: nickName || '',
    avatarUrl: avatarUrl || '',
    rankScore: 0,
    rankName: '初级小六',
    totalGames: 0,
    wins: 0,
    losses: 0,
    draws: 0,
    winRate: 0,
    maxRankScore: 0,
    energy: 30,
    energyMax: 30,
    lastSignInDate: '',
    signInStreak: 0,
    regretCards: 0,
    renameCards: 0,
    settings: { soundEnabled: true, vibrationEnabled: true, musicEnabled: true },
    createdAt: now,
    updatedAt: now,
    lastLoginAt: now,
  };
}

```